Lab 2.a — Writing the Standards Down

Skill: authoring the governance documents an AI agent actually uses — and deciding when each one is warranted. **Duration:** ~2 hours, facilitated. **Surface:** your own copy of this repo.

The challenge. This repo has a real, load-bearing rule that nothing explains and nothing checks. Document the *why* as an ADR (Architecture Decision Record), derive a rule from last week's real fixes and write it up as an NFR (Non-Functional Requirement), and wire both into CLAUDE.md so Claude actually reads them.

Why this block exists

Last week you fixed real bugs with Claude, test-first. It worked because this repo already told Claude how it works — the stack, the conventions, the commands, where things live.

You have sixteen developers and, right now, sixteen private versions of that context. The habits are good and none of them are shared. This block turns habits into standards: written down, reviewed, and pointed at from the one file that is always in context.

One finding worth knowing before you start: an ETH Zurich study published in February 2026 found that **LLM-generated context files made task success slightly worse** than providing no context at all (about −3%), while raising cost by more than 20%. Human-written files improved success (about +4%). Running `/init` and walking away is worse than doing nothing. You will use Claude to draft — from evidence in the code — and you will own the decisions. *(Study summary: "CLAUDE.md vs AGENTS.md vs SKILL.md (2026)" — ask in Slack for the paper link.)*

Before the session

- [ ] Your copy of this repo runs: `npm install`, `npm run reset-db`, `npm run dev`, `npm test` green
- [ ] A clean branch: `git checkout -b feat/governance` — today's work ships as PRs from it
- [ ] Your Lab 1.b PR link handy — we're going to look at real fixes from last week, possibly yours
- [ ] Skim `docs/adr/0001-shared-api-response-envelope.md` and `docs/nfr/0001-external-api-error-handling.md` — one worked example of each form ships in this repo
- [ ] Pull the lab materials into your copy (they didn't exist when your copy was created):

```
npx giget gh:mike-tajmajer-fullstacklabs/fsl-taskboard-lab/docs/labs docs/labs
```

- [ ] Optional, keeps the `Closes #N` habit from Lab 1 — seed the Lab 2 backlog into your copy (Windows: run it in **Git Bash**, which ships with Git for Windows):

```
bash docs/labs/snippets/seed-lab2-issues.sh
```

> Why the pull is needed: your copy was made from the template before these lab materials existed, and template copies don't inherit later changes. That is why things arrive as snippets you copy in rather than files already in your repo — and it's a preview of Lab 2.b's last topic: how a standard travels to repos that already exist. (One snippet — 2.b's settings block — must be **merged** into an existing file, never pasted over it; the handout says so loudly when you get there.)

The five artifact types

Three words, and how they relate — everything today hangs on this: a **rule** is what the team agrees on; a **document** is where the rule lives (one document can hold many rules); a rule written down and agreed is a **standard**. The five artifact types below are five *kinds of document*, distinguished by the question each answers.

Two acronyms too, since everything below leans on them: an **ADR** is an *Architecture Decision Record* — one decision, written down with its reasons and costs. An **NFR** is a *Non-Functional Requirement* — a rule about *how* the system must behave (logging, safety, error handling) rather than *what* it does.

Each artifact answers a **different question**. That's the cleanest way to keep them apart — if you can name the question, you know which document you're writing.

Artifact	The question it answers	What it is
CLAUDE.md	"What must the agent know before anything else?"	Always-loaded context + pointers
ADR	"**Why** is it like this?"	One decision, with its cost
NFR	"**What must always be true**, and how do we check?"	A standing, testable rule
Best-practice recipe	"**How** do I do X here?"	A worked procedure — "recipe" from here on
Rules file (`.claude/rules/`)	"What applies **only** when touching these files?"	Path-scoped context

One subject, five artifacts

This repo has a real rule: **only `server/src/repositories/` may import `db/store.ts`.** Here is what each artifact says about that same subject — and notice that none of them is redundant:

Artifact	What it says about the boundary
CLAUDE.md	One line, no rationale: `Don't: access db.json outside server/src/repositories/`. Always loaded
ADR	Why a repository layer was chosen, what else was considered, and what it costs. Written once
NFR	The rule as a standing requirement — **plus how compliance is checked**
Recipe	`docs/best-practices/adding-a-resource.md` — the steps for adding a resource *within* that boundary
Rules file	Guidance that loads only when you're in the files it applies to

The ADR is **history** — why we chose this, once, and what we gave up. The NFR is **law** — what must hold now, and how you'd know. The recipe is **procedure**. CLAUDE.md is the **index**. The rules file is **context that arrives just in time**.

Telling them apart when it's genuinely unclear

All five, on the axes that actually differ:

	CLAUDE.md	Rules file	ADR	NFR	Recipe
Answers	What to know first	What applies *here*	**Why**	What must hold	**How**
Tense	Present, standing	Present, scoped	Past — a choice made	Present — holds now	Imperative
Lifecycle	Revised continuously	Revised in place	**Superseded**, never edited	Revised in place	Updated freely
Carries a check?	No	No	No	**Yes — that's why**	No
When it loads	**Always**	When you touch matching paths	On demand, via a trigger	On demand + a binding line	On demand, via a trigger
Lose it and…	The agent is unoriented	Guidance stops arriving when needed	You lose the *why*	You lose the *rule*	You lose the *how*

Two rows do most of the work. **Carries a check?** separates the NFR from everything else — it is the only artifact that obliges you to say how compliance is verified, which is why it's the one that leads into guardrails. **When it loads** separates the two context files from the three documents: CLAUDE.md is always there and therefore expensive, a rules file arrives only where it applies, and everything else waits to be fetched.

Quick test. One-time choice → ADR. Must always hold and you can say how it's checked → NFR. Steps someone follows → recipe. Needed every session → CLAUDE.md. Only relevant in some files → rules file.

The pair people conflate — NFR vs recipe: an NFR **binds** (what must be true, checkable; a deviation is a defect). A recipe **guides** (the recommended route there; deviate freely, so long as the NFRs still hold). This repo's own pair: the envelope rule is an NFR — a bare `res.json()` is a defect; `adding-a-resource.md` is a recipe — you may add a resource a different way and violate nothing.

Do ADRs and NFRs come in pairs? No, and no. An NFR needs an ADR behind it only when the rule was a *contested choice* — NFR-0001 (external-API errors) has no ADR, correctly: it's hygiene, not a decision. An ADR spawns an NFR only when the decision leaves a *rule you can check per change* — a cloud-provider choice constrains everything and checks nothing. They pair when one subject has both, like the store boundary you'll work on today: ADR-0002 records *why* the layer exists; the boundary NFR states *what must hold* and how it's checked. When both exist they **cross-reference, never duplicate** — the NFR says "rationale: ADR-0002", the ADR says "enforced via NFR-0002". One home per fact.

One home per fact

The failure mode is writing the same content into all five. Then the copies stop matching — **drift**, the same word 2.b uses for copies across repos — and nobody knows which one is authoritative. **Each fact lives in exactly one place; the others point at it.** That is why CLAUDE.md has a trigger table instead of the docs' contents.

The scope hierarchy is worth holding onto too — a **layer** is just a rule's *reach*: who has to obey it. Everyone in every repo → **Collective** ("never commit secrets"). Everyone in one repo → **Project** ("responses only via the respond.ts envelope"). Only work touching certain files → **Feature/path** ("server tests must use makeTestDb()" — .claude/rules/testing.md). The reach decides where the document lives, so the right people and agents inherit it. Most teams have no Collective layer and an accident of a Project one — notice this repo's secrets rule is Collective-reach but parked in a Project file, because no org-wide home exists yet.

There is a **sixth artifact**, and it's the only one that lives at the Collective layer: *how a standard reaches every repo, and how anyone knows they're current.* That one is a **process** doc your team defines rather than something an instructor hands over — Lab 2.b hands it to you, and `distributing-standards.md` has the options and a template. Worth knowing it exists now, because everything you write today will eventually need to travel.

What you'll do

1. Watch one ADR get written — then write yours

Your instructor documents the repository-pattern boundary live: the gap, the evidence in the code, Claude drafting the Context, a human owning the Decision, commit.

Then **you write the same ADR in your own copy** — same subject, your own words, ~10 minutes. Use the prompt starters below; the point is the drafting motion, not novelty. The whole demo is written down step-by-step with worked examples in `lab-2a-demo-walkthrough.md` — replay it after class, or use it to catch up if you missed the session.

Checkpoint: you have an ADR staged in `docs/adr/` with the Status/Date/Deciders block completed and all three sections — Context, Decision, Consequences — filled.

> **Whose work is what, once, clearly:** everything in this lab is authored by **you, in your own copy** — the clause list in step 2, the ADR here, the NFR in step 3. The only shared step is the short class compare after step 2, where the lists go side by side and the class agrees one **AGREED** list — calibration, not committee work. (Missed the session or working solo? The lab works end-to-end alone, by design — `lab-2a-derive-walkthrough.md` is the step-by-step guide for exactly that. Don't open it before the live session; it replays the discovery.)

2. Derive a rule from real code

We put three fixes for the same bug on screen — authored for this exercise. (Your own 1.b PRs all did it right; fix C is that discipline. A and B are what happens on real teams on busy days.) You read them cold and answer one question:

> **Which of these are acceptable? Write the rule.**

You produce your own list of candidate clauses — on your own, no conferring yet. Start from the four seed clauses: verdict each one **keep / tighten / drop**, then add your own, and mark the one you're **least sure** about. A clause earns **keep** only if you can name **which of A / B / C it catches**. *Tighten* means more checkable, not more strict. A good clause names an observable fact — "a change under `server/src` ships with a test in the same PR" is a clause; "tests should be good" is a wish.

For every clause, tag it:

- **mechanical** — a script, given only the diff, could return pass/fail
- **fuzzy** — it needs a person (if you need to know *intent*, it's fuzzy)

That tag is the whole point. Do not skip it.

Then the compare: paste your list into the shared Doc under your name. The class puts the lists side by side and agrees **one AGREED list** — where the lists disagree is where the standard is actually being made, so the least-sure clauses get read out loud. Keep your own verdicts visible; the compare is calibration, not a vote to erase them.

Where this lands: the AGREED clauses become your **NFR** in step 3. Why an NFR and not an ADR? Apply the quick test from earlier: this list must *always hold* and says *how it's checked* — present-tense law, revised in place. (If adopting the rule ever becomes genuinely contested, *that* argument would earn an ADR. The know-how — "how do I write a failing test first?" — is recipe material.)

3. Write it up as an NFR

Turn the AGREED clauses into a doc in `docs/nfr/` — your wording, your document. If your list disagreed with the class list, AGREED is still what you write up; your dissent goes in as a **dropped clause with a reason** — that's how disagreement with a live standard is recorded on a real team. Copy the *headings and tone* of NFR-0001 — but note that NFR-0001 predates one thing you're being asked for: **per-clause checks**. Your doc adds a clause list shaped like this:

```
## The rules

1. A change under `server/src/**` ships with a test in the same PR.
   **Checked by:** CI — the diff must touch a `*.test.ts` when it touches
        `server/src`.
2. The test must have failed before the fix.
   **Checked by:** — NOT MECHANICALLY CHECKABLE — decision: human gate (reviewer
        asks) — <name>
```

That second line is the **marking convention**: every clause states how compliance is checked, and a clause that *can't* be checked is marked, carries a decision (accepted risk · human gate · dropped), and a name. An unmarked unenforceable clause is the dangerous kind.

Checkpoint: your NFR names how each clause is checked — and which ones can't be.

4. Wire both docs into CLAUDE.md

A doc nothing points at is a doc nobody reads, Claude included. Two kinds of wiring — and they're the two loading modes from Lab 1 (pointer = **lazy**, `@import` = **eager**), each used where it earns its cost:

1. **A trigger row** per doc, in the "Reference docs — read on demand" table — phrased as a situation the reader is *in* ("Changing X → read Y"). This is Lab 1's lazy pointer with its firing condition made explicit: the pointer is *how* the doc loads; the trigger is *when*.
2. **A binding line** for the NFR: a deliberate, one-line **eager** load — the rule's core in CLAUDE.md's Do/Don't section, in force *every* session without being fetched. Eager costs every session, which is why it gets one sentence and the trigger row gets the document. Concretely:

```
- Do: every change under server/src ships with a test in the same PR (NFR-0002)
```

The doc is the law; the binding line is the sign on the wall. (This repo doesn't have one for NFR-0001 yet — you're setting the pattern.)

Then **smoke-test it**: start a fresh session, ask Claude the rule, and confirm it reads your doc rather than guessing.

Prompt starters

Swap in your own paths. Note what each one does *not* do: none of them ask Claude to decide.

Step	Prompt
Gather evidence	`Read @server/src/repositories/ and @server/src/services/todos.service.ts. What boundary do these files enforce, and what would break if it were violated? Don't propose changes.`
Draft an ADR	`Using @docs/adr/template.md, draft an ADR for that boundary. Fill Context and Consequences from what you just read. Leave Decision as a question for me to answer.`
Pressure-test a clause	`Here is a draft rule: "<clause>". Could a script decide whether a change complies, using only the diff? If not, say exactly what a human has to judge.`
Draft an NFR	`Using @docs/nfr/0001-external-api-error-handling.md as the form, turn these clauses into an NFR. For each clause add how compliance is checked. Mark any you cannot express as a check.`
Wire it in	`Add a row to the reference-docs table in @CLAUDE.md for @docs/nfr/<file>.md. Phrase the trigger as a situation, matching the existing rows.`

How it's assessed

Lab rubric + peer review, same as Lab 1: **Completion · Quality · Process · Independence · Insight**, each 1–4. There is deliberately no answer key for your NFR — Process (did you separate mechanical from fuzzy?) and Insight (can you say *why* a clause resists checking?) carry the weight. This lab counts toward the **Agentic Developer** certification at the week-4 gate.

Acceptance criteria

- [ ] One **ADR** committed in `docs/adr/`, following the template, with at least one honest cost in Consequences
- [ ] One **NFR** committed in `docs/nfr/`, every clause stating how it is checked
- [ ] Clauses that can't be mechanically checked are **marked**, each with a recorded decision
- [ ] Both docs **wired into CLAUDE.md** — trigger-table rows plus one binding line — and smoke-tested in a fresh session
- [ ] Everything reviewed by a teammate via PR, CI green
- [ ] A **reflection note** — 3–5 sentences in your PR description: what you learned, what surprised you, what you'd apply at work tomorrow

Stretch

- Derive clauses for the **second convention** — the store-boundary rule (only `server/src/repositories/` may import `db/store.ts`); it's the documented-but-unchecked case

- The **half-gate case**: `console.log` in `server/src` fails lint, but `logger.info('wrong-first-arg', …)` passes silently. Write the clause that closes the gap, and tag it
- Convert a prose rule into a path-scoped rules file, following `.claude/rules/testing.md`
- Review a peer's ADR against the fuller MADR fields: Decision Drivers, Considered Options
- Get a head start on the "how we use these" README — when each artifact is warranted, who authors it, who reviews it, where it lives. It's a required deliverable in 2.b, and today's session is when you know the answers

Common stuck points

Symptom	What to do
"I don't know what decision to write an ADR about"	Use the trigger: hard to reverse, contested, or a newcomer would ask why. The store boundary qualifies on all three
The NFR reads like a wish	If you can't say how it's checked, it isn't an NFR yet. Write the check first, then work backwards to the wording
Claude wrote the whole doc and it looks fine	Read it as someone who disagrees. If there's nothing to disagree with, it's a description, not a decision
Your list disagrees with the AGREED list	Good. Write the AGREED version, record yours as a dropped clause with a reason — that sentence is more valuable than the clause
"This rule can't be enforced, so why write it?"	Unenforceable rules still coordinate people. What they must not do is *pretend* to be enforced — hence the marking
"Why write an ADR for a decision already made?"	Recording is not deciding — a retroactive ADR captures the *why* before it evaporates. Status: Accepted; Date: when recorded
"What if I disagree with the decision?"	Record it anyway (it *is* this repo's reality) and put your objection in Consequences as a cost. Recording ≠ endorsing
Every clause ends up "Checked by: code review"	That's the human gate as a reflex, not a decision. For each one, ask once: could a script check *this* from the diff?

Where this goes next

Lab 2.b takes the clause you derived and makes something refuse to violate it — then asks the harder question: how does a standard reach every repo you already have?